

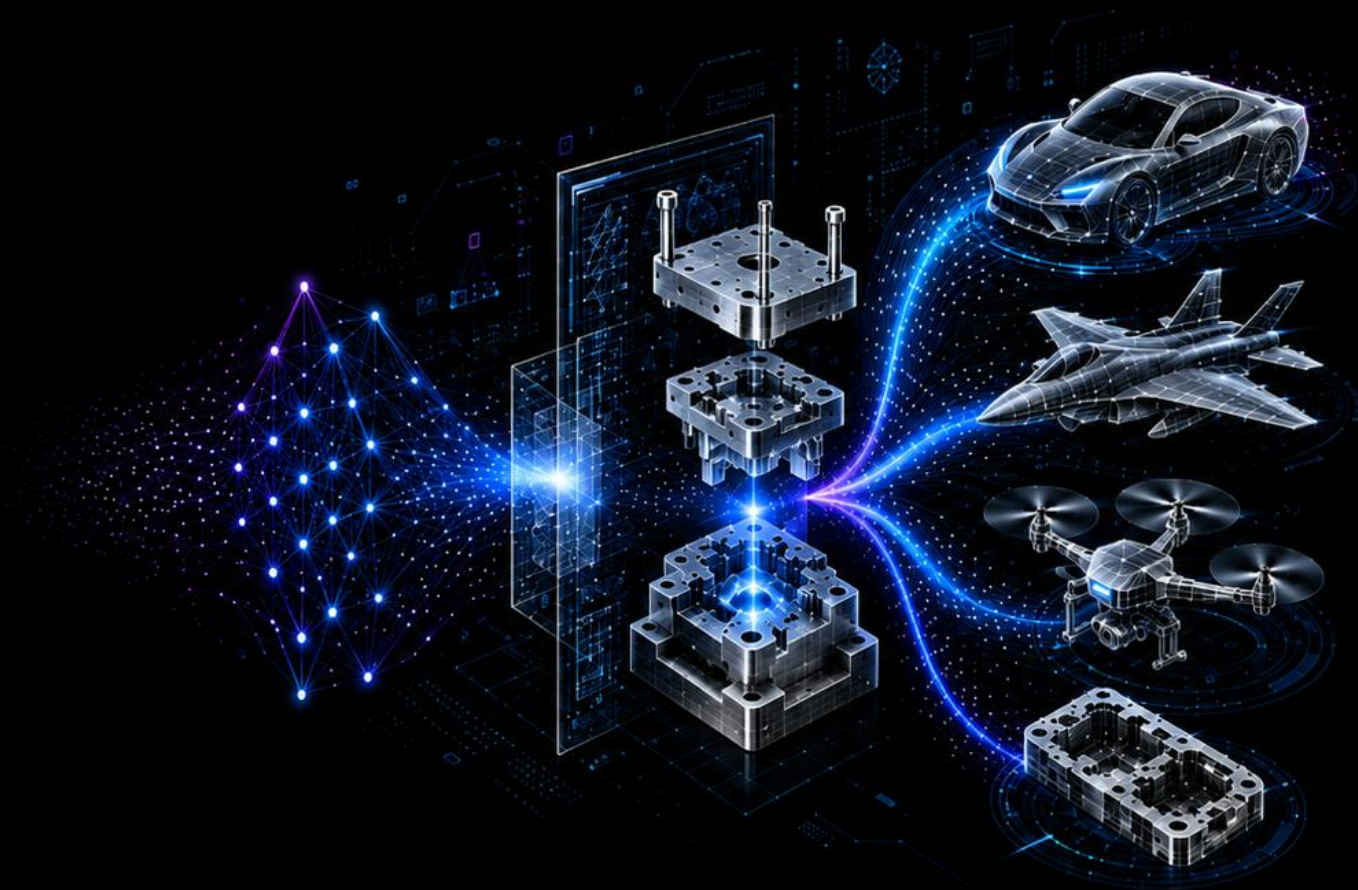
정규 스터디

# 7장 시계열분석 (코드 중심)

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design  
for Engineering  
Applications Lab

## 발표 목적과 전체 실험 흐름

- RNN, LSTM, GRU 예제 코드를 직접 실행하고 수정 전후의 결과를 비교

### 발표 순서

#### ① 교재 원본 코드 재현

교재에 제시된 RNN과 LSTM 코드를 동일한 조건으로 실행하고, GRU는 LSTM과 동일한 SBUX 데이터와 학습 조건으로 구현하였다.

#### ② 코드 수정 전후 비교

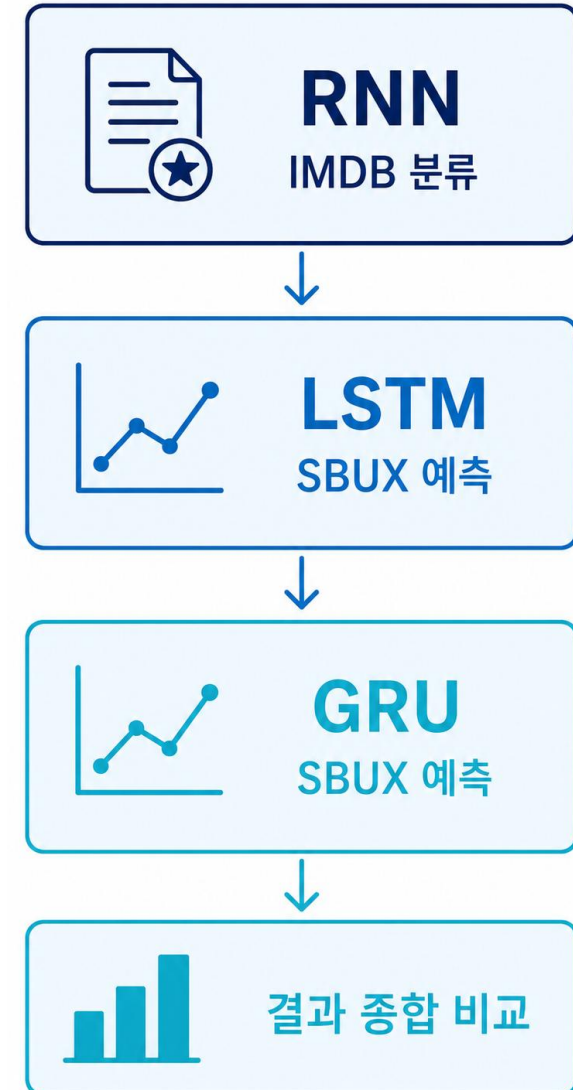
RNN에서는 출력층의 sigmoid를 제거하였다. LSTM과 GRU에서는 난수 seed를 고정하고 학습 중 test loss를 측정하는 코드를 추가하였다.

#### ③ 실행 결과 분석

코드 수정이 실제 성능을 개선했는지 확인하고, train loss만으로는 발견하기 어려운 과적합과 예측 곡선의 한계를 분석하였다.

#### ④ 세 모델의 차이 정리

동일한 SBUX 데이터로 실행한 LSTM과 GRU의 파라미터 수, 학습 시간과 loss를 비교하였다.



## ■ 실험 데이터와 실행 환경

- RNN은 IMDB 감성분류, LSTM과 GRU는 SBUX 거래량 예측에 적용

### RNN 실험

#### 데이터 구성

- ✓ Dataset: IMDB 데이터
- ✓ Train 원본: 25,000개
- ✓ Test: 25,000개
- ✓ Train의 20%를 valid\_data으로 분리
- ✓ 입력: 영화 리뷰 텍스트
- ✓ 정답: 긍정 또는 부정

#### 학습 조건

- ✓ 최대 시퀀스 길이: 200 tokens
- ✓ Epoch: 5
- ✓ Batch size: 100
- ✓ Loss: Cross Entropy
- ✓ 평가 지표: Accuracy

### LSTM·GRU 실험

#### 데이터 구성

- ✓ Dataset: Yahoo Finance SBUX CSV
- ✓ 기간: 2019-12-11~2020-12-10
- ✓ 전체 데이터: 253거래일
- ✓ Train: 앞의 200일
- ✓ Test: 이후 53일
- ✓ 입력: Open, High, Low, Close, Volume
- ✓ 예측 대상: Volume

#### 공통 학습 조건

- ✓ Hidden size: 2
- ✓ Epoch: 1,000
- ✓ Learning rate: 0.0001
- ✓ Loss: Mean Squared Error
- ✓ 실행 환경: 로컬 CPU



### IMDB REVIEW

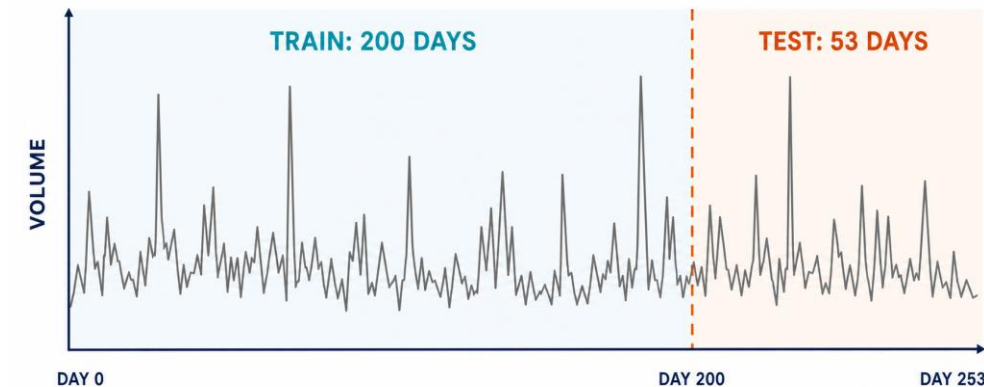
“

The movie was entertaining and the acting was excellent.

”

**LABEL: POSITIVE**

### SBUX VOLUME DATA



## RNN — 교재 원본과 sigmoid 제거

### 교재 원본

[교재 예제 그대로 sigmoid 사용]

```
python
def forward(self, x):
    ...
    logit = torch.sigmoid(
        self.out(h_t)
    )
    return logit

loss = F.cross_entropy(logit, y)
```

- ✓ 코드 설명: 교재 원본은 출력층에서 sigmoid를 적용한 뒤 그 값을 cross\_entropy()에 전달한다.
  - ✓ 문제점: PyTorch의 Cross Entropy는 sigmoid나 softmax를 적용하지 않은 raw logits를 입력받도록 설계되어 있다.
- ⇒ 이미 sigmoid를 통과한 값은 0~1 범위로 제한되며, Cross Entropy 내부 연산과 중복되어 학습 신호를 약하게 만들 가능성이 있다.

### 수정 코드

[sigmoid 제거]

```
python
def forward(self, x):
    ...
    logit = self.out(h_t)
    return logit

loss = F.cross_entropy(logit, y)
```

- ✓ 코드 설명: 두 번째 실행에서는 출력층의 sigmoid만 제거하고, self.out(h\_t)의 raw logits를 Cross Entropy에 직접 전달하였다.
- ✓ 동일하게 유지한 조건
  - Dataset: IMDB
  - Sequence length: 최대 200 tokens
  - Epoch: 5
  - Batch size: 100
  - RNN 구조와 Hidden size
  - Optimizer와 Learning rate

## RNN 결과 — sigmoid 제거만으로는 학습이 개선되지 않음

- 올바른 코드 수정이었지만 Test Accuracy는 여전히 50%에 머물렀다

### sigmoid 존재 (기존)

#### ✓ 실행 결과

Validation Accuracy: 49~50%

Validation Loss: 약 0.69

총 실행 시간: 426.5초

5 Epoch 동안 정확도 변화 거의 없음

⇒ 1차 실행에서는 Validation Accuracy가 무작위 추측 기준선인 50% 근처에서 벗어나지 못했다. Validation Loss도 약 0.69에서 거의 변화하지 않았다.

### sigmoid 제거 (수정)

#### ✓ 실행 결과

Validation Accuracy: 50~51%

Validation Loss: 약 0.69~0.70

총 실행 시간: 468.8초

1차 실행과 의미 있는 차이 없음

⇒ Sigmoid를 제거한 뒤에도 Validation Accuracy는 최대 51%에 그쳤고, 최종 Test Accuracy는 50%로 동일했다. Test Loss 역시 0.69에서 0.70으로 소폭 증가했다.

#### 1차: sigmoid 사용

50.0%

Test Accuracy

0.69

Test Loss

sigmoid 제거

#### 2차: sigmoid 제거

50.0%

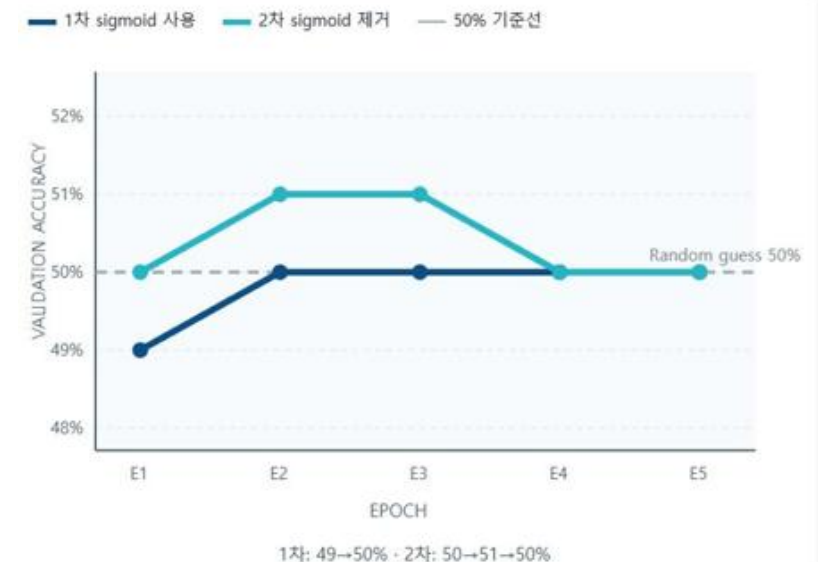
Test Accuracy

0.70

Test Loss

#### 가설 기각

올바른 코드  
수정이었지만 성능은  
사실상 동일했다.



## LSTM — 교재 원본 코드 실행

- 교재의 모델 구조와 하이퍼파라미터를 유지하고 SBUX 거래량 데이터로 학습

### 원본 학습 코드

#### ✓ 교재 설정

Hidden size: 2 /Epoch: 1,000 /Learning rate: 0.0001

/Loss function: MSE / Train·Test: 200일·53일

```
python
for epoch in range(num_epochs):
    outputs = model(X_train)

    optimizer.zero_grad()

    loss = criterion(
        outputs,
        y_train
    )

    loss.backward()
    optimizer.step()

    if epoch % 100 == 0:
        print(
            epoch,
            loss.item()
        )
```

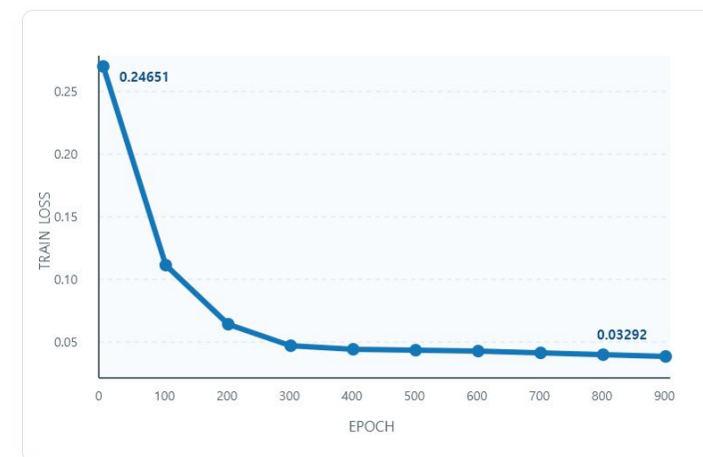
- ✓ **설명:** 원본 코드는 100 epoch마다 train loss를 출력한다. 학습하지 않은 Test 구간에 대한 loss는 별도로 계산하지 않는다.
- ✓ **한계:** Train loss만으로는 모델이 새로운 데이터에서도 계속 좋아지고 있는지 확인할 수 없다.

### 원본 실행 결과

#### ✓ Train Loss 변화

#### LSTM 교재 원본 — Train Loss

hidden\_size=2 · 1000 epochs · learning\_rate=0.0001



**0.24651**

Initial Train Loss

**0.03292**

Final Train Loss

**0.9 s**

Training Time

Loss의 대부분은 epoch 200 이전에 감소했다. 이후에는 개선 폭이 매우 작았다.

- ✓ **그래프 설명:** Train loss는 1,000 epoch 동안 0.24651에서 0.03292로 감소했다. 전체 감소의 대부분은 epoch 200 이전에 나타났고, 그 이후에는 매우 완만하게 개선되었다.

## LSTM — Seed 고정과 Test Loss 추적 추가

- 모델 구조는 유지하고 재현성과 일반화 성능을 확인하는 코드만 추가

### 추가한 코드

- ✓ 재현성을 위한 Seed 고정

```
python

device = torch.device(
    'cuda:0'
    if torch.cuda.is_available()
    else 'cpu'
)

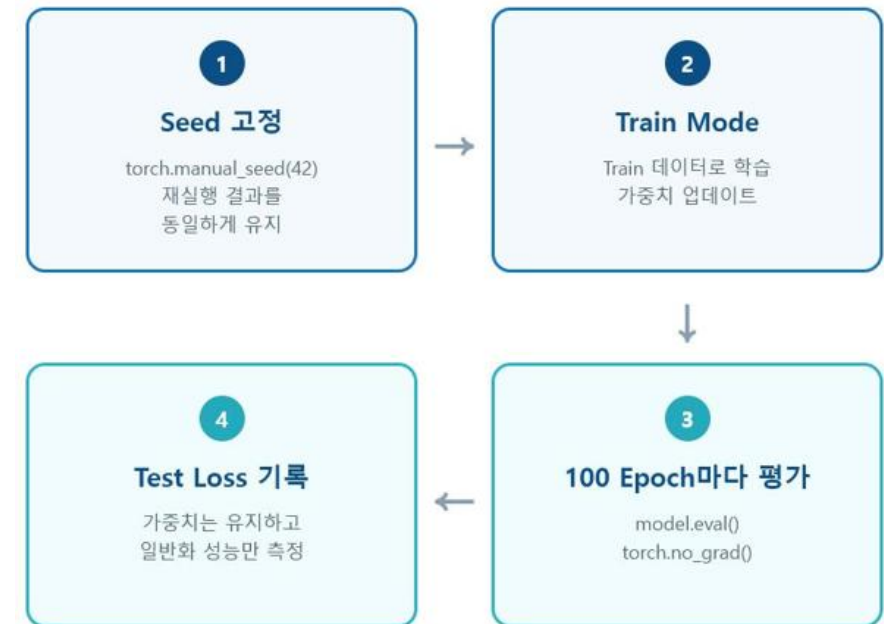
torch.manual_seed(42)
```

- ✓ 설명:

LSTM과 Linear 레이어의 초기 가중치는 실행할 때마다 무작위로 설정된다. 특히 hidden size가 2인 작은 모델에서는 초기값에 따라 결과가 크게 달라질 수 있어 seed를 42로 고정.

### 코드 작동 과정

- ✓ Test Loss 추적 과정



- ✓ 설명: Test Loss 계산은 가중치를 변경하지 않는다. 현재 모델이 학습하지 않은 미래 구간에서 얼마나 틀리는 지만 측정한다.

## LSTM 결과 — Epoch 300 이후 과적합 확인

- Train Loss는 계속 감소했지만 Test Loss는 최저점을 기록한 뒤 다시 증가

### LSTM Train Loss vs Test Loss

Test Loss는 epoch 300에서 최저점을 기록한 뒤 다시 증가했다.



**0.02869**

Final Train Loss

**0.01795**

Final Test Loss

**0.00974**

Minimum Test Loss

**E300**

Best Epoch

과적합 확인: E300 이후 Train Loss는 계속 감소했지만 Test Loss는 0.018~0.022까지 다시 증가했다.

### 결과 해석

#### [결과에서 확인한 내용]

⇒ Test Loss는 초기 0.25261에서 빠르게 감소하여 Epoch 300에서 0.00974의 최저값을 기록하였다.

그러나 Epoch 300 이후에도 학습을 계속하자 Train Loss는 지속해서 감소했지만, Test Loss는 다시 증가하였다.

최종 Epoch에서 Train Loss는 0.02869, Test Loss는 0.01795로 측정되었다. 따라서 학습 데이터에 대한 오차 감소가 반드시 새로운 데이터의 예측 성능 향상을 의미하지는 않는다.

#### [핵심 해석]

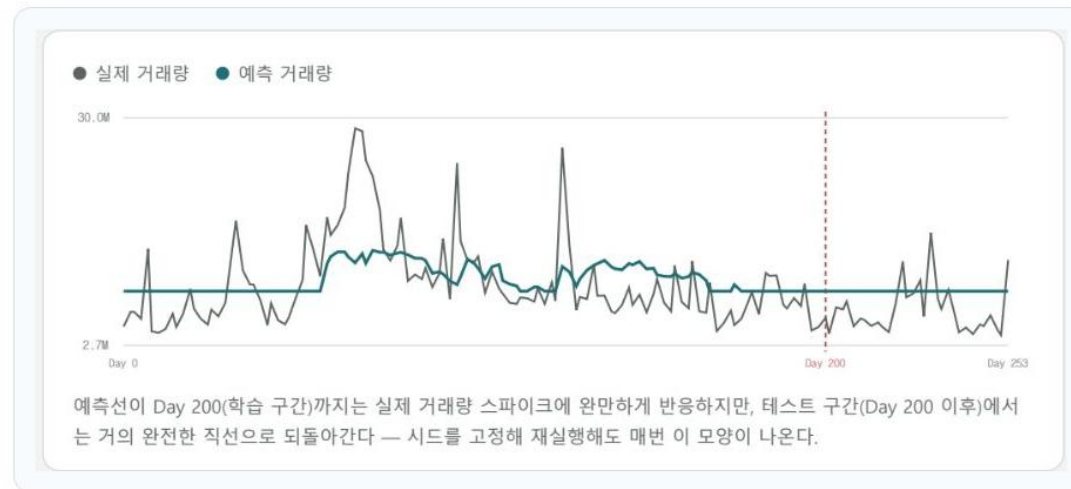
⇒ Epoch 300 이후의 추가 학습은 Train Loss만 감소시켰으며, Test Loss 기준 일반화 성능은 오히려 악화되었다. 이는 과적합이 시작되었다는 근거로 해석된다.

## LSTM 예측 — 학습 구간에는 반응했지만 테스트 구간은 평탄화

- 실제 거래량과 예측 거래량을 비교하여 모델이 어떤 패턴을 학습했는지 확인하였다.

### LSTM 예측값과 실제 거래량

학습 구간의 큰 변화에는 반응했지만, 테스트 구간에서는 평균값에 가까운 직선으로 수렴했다.



TRAIN · Day 0-199

TEST · Day 200-252

#### 학습 구간

거래량 스파이크에 완만하게 반응

#### 테스트 구간

약 7-9M 부근의 평평한 값으로 수렴

#### 입력 구조

각 샘플의 sequence length가 1

**핵심 해석:** LSTM 셀의 기억 능력보다, 과거 흐름을 시퀀스로 제공하지 않은 데이터 구성이 예측을 제한했다.

### 그래프 해석

#### [예측 형태에서 확인한 한계]

⇒ 학습 구간에서는 실제 거래량이 크게 증가하는 구간에 예측 값도 완만하게 반응하였다. 따라서 모델이 **전체적인 거래량 수준과 일부 변화**는 학습한 것으로 볼 수 있다.

하지만 Day 200 이후 테스트 구간에서는 예측 값이 약 7~9M 부근의 평평한 값으로 수렴하였다. 실제 거래량의 상승과 하락을 세밀하게 따라가지 못하고 **평균적인 값을 출력한 것**이다.

#### [핵심 해석]

⇒ LSTM을 사용했다는 사실만으로 시간의 흐름이 자동으로 학습되는 것은 아니다.

## GRU — 교재 원본에 Test Loss 추적 추가

- 모델 구조와 학습 조건은 유지하고, 재현성과 일반화 성능을 확인하는 코드만 추가

### GRU 교재 원본과 평가 코드 추가

모델 구조와 학습 조건은 유지하고, 재현성과 일반화 성능 관찰 기능만 추가했다.

#### ① 교재 원본

```
model = GRU(
    num_classes=1,
    input_size=5,
    hidden_size=2,
    num_layers=1
)

for epoch in range(1000):
    ...
    loss.backward()
    optimizer.step()
```

253일  
SBUX 거래일

#### ② 수정 버전

```
torch.manual_seed(42)

if epoch % 100 == 0:
    model.eval()
    with torch.no_grad():
        test_loss = criterion(
            model(X_test), y_test
        )
    model.train()
```

200 / 53  
Train / Test

1000 epochs  
lr = 0.0001

### 원본 구조 유지 → Test Loss 관찰 가능

변경하지 않은 것: GRU 구조, hidden\_size, 데이터 분할, epoch 수, 학습률  
 추가한 것: 난수 시드 고정과 100 epoch 간격의 Test Loss 측정

### 실험 방법

#### [교재 원본 실행 조건]

- ✓ LSTM 과 동일한 SBUX 거래량 데이터를 사용하였다.  
전체 253거래일 중 앞의 200일은 학습 데이터, 이후 53일은 테스트 데이터로 분리하였다.
- ✓ GRU 모델은 교재와 동일하게 input\_size=5, hidden\_size=2, num\_layers=1로 구성하였다. 학습률은 0.0001이며 총 1,000 epoch 동안 학습하였다.

#### [추가한 두 가지 기능]

- ✓ torch.manual\_seed(42)를 추가하여 초기 가중치를 고정했다.  
이를 통해 코드를 반복 실행해도 동일한 결과가 나오도록 하였다.
- ✓ 100 epoch마다 평가 모드로 전환하여 Test Loss를 계산했다. 이 과정은 모델 가중치를 변경하지 않고 현재 모델의 일반화 성능만 확인한다.

## GRU 결과 — Epoch 300 이후 완만한 과적합

- Train Loss는 계속 감소했지만 Test Loss는 Epoch 300 이후 조금씩 증가

### GRU Train Loss vs Test Loss

Test Loss는 epoch 300에서 최저점을 기록한 뒤 0.010~0.011 부근으로 완만하게 증가했다.



0.02022

Final Train Loss

0.01135

Final Test Loss

0.00987

Minimum Test Loss

E300

Best Epoch

완만한 과적합: E300 이후 Train Loss는 계속 감소했지만 Test Loss는 0.00987에서 0.01135로 서서히 증가했다.

### 결과 해석

#### [학습 과정에서 확인한 내용]

⇒ GRU의 Test Loss는 초기 0.18809에서 빠르게 감소하였다. Epoch 300에서 0.00987의 최저값을 기록했으며, 이후에는 0.010~0.011 사이에서 완만하게 증가하였다.

반면 Train Loss는 학습이 끝날 때까지 계속 감소했다. 최종 Train Loss는 0.02022, 최종 Test Loss는 0.01135로 측정되었다.

따라서 GRU에서도 Epoch 300 이후 학습 데이터와 테스트 데이터의 Loss 방향이 달라지는 과적합 현상이 나타났다.

#### [핵심 해석]

⇒ Epoch 300 이후 추가된 600 epoch의 학습은 Train Loss만 감소시켰다. Test Loss는 0.00987에서 0.01135로 증가했으므로 일반화 성능은 오히려 낮아졌다.

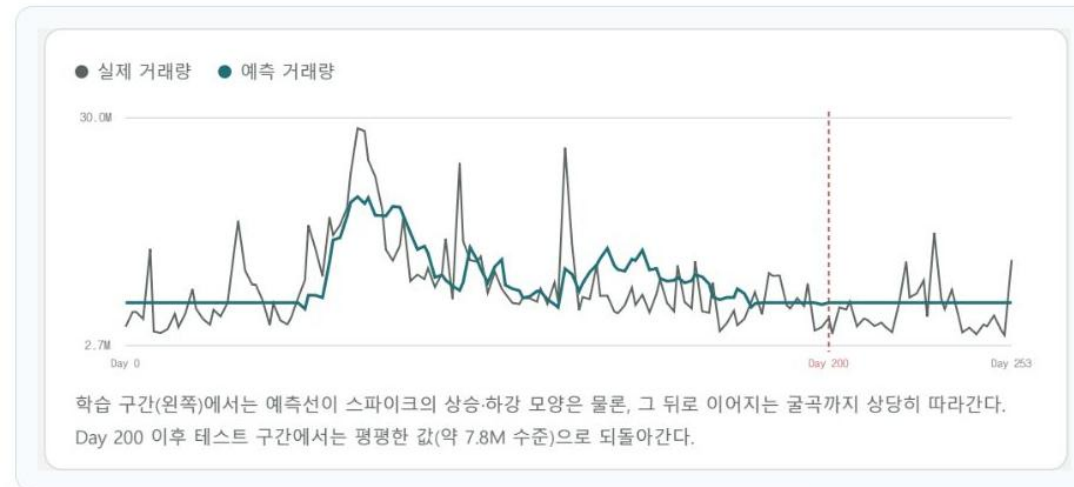
GRU도 LSTM과 동일하게 Epoch 300에서 가장 낮은 Test Loss를 기록했다. 다만 이후의 Test Loss 증가 폭은 LSTM보다 상대적으로 완만해졌다.

## GRU 예측 — 학습 구간은 세밀하지만 테스트 구간은 평탄화

- 실제 거래량과 GRU의 예측값을 비교하여 학습 구간과 테스트 구간의 차이를 확인

### GRU 예측값과 실제 거래량

학습 구간의 스파이크와 굴곡을 비교적 세밀하게 추종했지만, 테스트 구간에서는 평탄화되었다.



TRAIN · Day 0-199

TEST · Day 200-252

#### 학습 구간

상승·하락과 후속 굴곡까지 비교적 세밀하게 추종

#### 테스트 구간

약 7.8M 부근의 거의 일정한 값으로 수렴

#### LSTM 대비

학습 구간 추종은 더 세밀하지만 테스트 한계는 동일

**핵심 해석:** GRU의 간결한 게이트 구조도 길이 1 입력에서는 과거 흐름을 활용할 수 없어, 새로운 구간에서 평균값에 가까운 예측을 선택했다.

### 결과 해석

#### [예측 형태에서 확인한 특징]

⇒ GRU 예측 값은 학습 구간에서 실제 거래량의 큰 상승과 하락 뿐만 아니라, 스파이크 이후 나타나는 완만한 굴곡도 비교적 세밀하게 따라갔다.

그러나 Day 200 이후 테스트 구간에서는 약 7.8M 수준의 거의 일정한 값으로 수렴하였다. 학습 데이터에서는 세부 패턴을 학습했지만, 새로운 데이터에서는 평균에 가까운 안전한 값을 출력한 것이다.

#### [핵심 해석]

⇒ GRU의 간결한 게이트 구조도 시퀀스 길이가 1인 입력에서는 과거 여러 날의 흐름을 사용할 수 없다. 따라서 새로운 구간에서는 평균값에 가까운 예측으로 수렴

GRU는 학습 구간에서 LSTM보다 세부적인 굴곡을 잘 따라갔지만, 테스트 구간에서는 두 모델 모두 평탄한 예측을 보임.

## LSTM vs GRU — 같은 데이터·같은 조건의 직접 비교

- SBUX 데이터, Hidden Size, Epoch, 학습률과 난수 시드를 동일하게 설정

### LSTM vs GRU 직접 비교

같은 SBUX 데이터와 동일한 학습 조건에서 두 모델의 효율과 일반화 성능을 비교했다.

| SBUX 253일     | Train 200 / Test 53 | hidden_size = 2 | seed = 42 |
|---------------|---------------------|-----------------|-----------|
| 비교 항목         | LSTM                | GRU             |           |
| 셀 파라미터        | 72                  | 54 · 25% 적음     |           |
| 최종 Train Loss | 0.02869             | 0.02022         |           |
| 최종 Test Loss  | 0.01795             | 0.01135         |           |
| 최저 Test Loss  | 0.00974             | 0.00987         |           |
| 최저점 시점        | E300                | E300            |           |
| 학습 시간         | 0.9 s               | 0.8 s           |           |

#### 효율성

GRU는 파라미터가 25% 적고 학습 시간도 약간 짧았다.

#### 최저 성능

최저 Test Loss는 LSTM이 0.00013만큼 낮았다.

#### 공통점

두 모델 모두 E300 이후 과적합과 테스트 예측 평탄화가 나타났다.

**판정:** 이번 단일 데이터셋에서는 GRU가 더 효율적이고 최종 Loss도 낮았지만, 최적 시점의 Test Loss는 LSTM이 근소하게 낮았다. 따라서 한 번의 실험만으로 어느 모델이 항상 우수하다고 일반화할 수 없다.

### 비교 결과 해석

#### [비교 결과]

⇒ GRU의 셀 파라미터는 54개로 LSTM의 72개보다 25% 적었다. 학습 시간도 GRU가 0.8초, LSTM이 0.9초로 GRU가 조금 더 짧았다.

최종 Train Loss와 Test Loss는 모두 GRU가 낮았다. 하지만 각 모델의 가장 좋은 시점을 비교하면 LSTM의 최저 Test Loss가 0.00974로, GRU의 0.00987보다 근소하게 낮았다.

두 모델 모두 Epoch 300에서 가장 낮은 Test Loss를 기록했으며 이후에는 과적합이 나타났다.

#### [핵심 해석]

⇒ GRU는 계산 효율에서 장점이 있었고, LSTM은 최적 시점의 Test Loss가 아주 조금 더 낮았다.

따라서 모델 선택은 정확도 뿐 아니라 파라미터 수와 계산 비용도 함께 고려해야 한다.

## Discussion — 모델 구조와 데이터 파이프라인은 다른 문제를 푼다

- 실험에서 확인된 실패 원인은 모델마다 서로 다른 입력 구조에서 발생

### 모델 구조와 데이터 파이프라인은 다른 문제를 푼다

이번 실험의 한계는 모델 이름보다 입력 시퀀스를 구성한 방식에서 더 선명하게 드러났다.

#### RNN · 긴 시퀀스 내부의 문제

최대 200 tokens → 마지막 상태



- sigmoid 제거 후에도 Test Accuracy 50%
- 반복되는 tanh 연산에서 기울기 소실 가능
- 리뷰 앞부분의 정보가 마지막 상태까지 유지되지 못함

다음 개선: IMDB 입력에 LSTM·GRU를 적용하고, 길이에 따른 성능 변화를 비교한다.

#### LSTM·GRU · 시퀀스를 안 보여준 문제

입력 Shape = (N, 1, 5)



- 게이트는 있지만 sequence length가 1
- 각 샘플마다 hidden state가 다시 초기화
- 테스트 구간에서 평균값에 가까운 평평한 예측

다음 개선: 과거 N일을 묶은 Sliding Window로 실제 시간 흐름을 입력한다.

**핵심:** 좋은 순환 셀을 선택하는 것만으로는 충분하지 않다. 모델이 기억할 수 있도록 데이터 자체를 올바른 시퀀스로 구성해야 한다.

### 종합 해석

#### [서로 반대되는 입력 문제]

⇒ RNN 실험에서는 최대 200개의 Token을 순서대로 입력하고 마지막 Hidden State로 감성을 분류하였다. 시퀀스가 지나치게 길어 앞부분의 정보가 마지막 상태까지 전달되지 못했을 가능성이 크다.

반대로 LSTM과 GRU 실험에서는 입력 형태가 (N, 1, 5)였다. 시퀀스 길이가 1이기 때문에 게이트 구조가 있어도 이전 날짜의 정보를 기억할 기회가 제공되지 않았다.

#### [모델 별 문제 정리]

##### ✓ RNN

문제: 긴 시퀀스 안에서 정보 소실

관찰: Sigmoid 제거 후에도 Test Accuracy 50%

추정 원인: 반복적인 tanh 연산에 따른 기울기 소실

##### ✓ LSTM·GRU

문제: 실제 시계열 시퀀스를 입력하지 않음

관찰: 테스트 구간에서 평평한 예측

추정 원인: 하루 단위 입력과 매 샘플 Hidden State 초기화

#### [결론]

⇒ LSTM이나 GRU처럼 좋은 순환 셀을 선택하는 것만으로는 충분하지 않다. 모델이 기억할 수 있도록 데이터를 올바른 시퀀스 형태로 구성해야 한다.

## Conclusion — 코드 수정, 평가 방법, 입력 구성의 영향

- 교재 예제 재현과 추가 실험을 통해 확인한 모델별 결과를 종합

### 모델별 최종 결론

#### ✓ RNN

Sigmoid 제거는 Cross Entropy가 Raw Logits를 입력받아야 한다는 점에서 올바른 수정이었다.

하지만 Test Accuracy는 여전히 50%였으며, 긴 IMDB 시퀀스에서 발생한 학습 정체를 해결하지 못했다.

#### ✓ LSTM

Test Loss를 추가로 추적하면서 Epoch 300 이후 과적합이 발생한다는 사실을 확인했다. 그러나 입력 시퀀스 길이가 1이어서 테스트 구간에서는 예측값이 평탄화되었다.

#### ✓ GRU

LSTM보다 적은 파라미터와 짧은 학습 시간으로 낮은 최종 Loss를 기록했다. 하지만 GRU 역시 길이 1 입력에서는 과거 흐름을 활용하지 못했다.

### Conclusion

코드 한 줄, 평가 루프, 입력 구성의 영향

|                                                                                                                                            |                                                                                                                                     |                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <p><b>R</b></p> <p><b>RNN</b></p> <p>Sigmoid 제거는 Cross Entropy 사용법에 맞는 수정이었지만, 긴 IMDB 시퀀스의 학습 정체는 해결하지 못했다.</p> <p><b>Accuracy 50%</b></p> | <p><b>L</b></p> <p><b>LSTM</b></p> <p>Test Loss 추적으로 E300 이후 과적합을 확인했다. 테스트 예측은 평균값 부근으로 평탄화되었다.</p> <p><b>Min Test 0.00974</b></p> | <p><b>G</b></p> <p><b>GRU</b></p> <p>더 적은 파라미터로 낮은 최종 Loss를 기록했다. 하지만 길이 1 입력의 한계는 해결하지 못했다.</p> <p><b>54 params · 0.8 s</b></p> |
|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|

### 이번 실험의 세 가지 교훈

|                                                                      |                                                                   |                                                               |
|----------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------|
| <p><b>1. 코드 수정</b></p> <p>이론적으로 올바른 한 줄 수정이 항상 성능 개선을 보장하지는 않는다.</p> | <p><b>2. 평가 방법</b></p> <p>Train Loss만 보면 최적 시점과 과적합을 놓칠 수 있다.</p> | <p><b>3. 입력 설계</b></p> <p>순환 셀보다 먼저 데이터를 실제 시퀀스로 구성해야 한다.</p> |
|----------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------|

### 다음 실험

N-day Window → Validation Split → Early Stopping

**최종 결론:** 모델 이름만 바꾸는 것보다, 올바른 출력·일반화 지표·시퀀스 입력을 함께 설계하는 것이 성능 개선의 출발점이다.

### 추후 방향성

#### ✓ 깨달은 점

1. 코드 수정  
이론적으로 올바른 한 줄의 수정이 항상 성능 향상을 보장하지는 않는다.

2. 평가 방법  
Train Loss만 확인하면 최적 학습 시점과 과적합 발생 여부를 놓칠 수 있다.

3. 입력 설계  
순환 셀의 종류를 바꾸기 전에 데이터를 실제 시퀀스 형태로 구성해야 한다.

#### ✓ 결론

⇒ 이번 실험을 통해 모델 구조의 차이도 중요하지만, 모델에 어떤 형태의 데이터를 입력하고 어떤 지표로 평가하는지가 실제 결과에 더 큰 영향을 줄 수 있다는 것을 확인했다.

# Any Questions?

이름 최준석  
소속 한양대학교 기계공학과  
이메일 [chw10109@hanyang.ac.kr](mailto:chw10109@hanyang.ac.kr)



*Intelligent Design  
for Engineering  
Applications Lab*

